

Model Evaluation Sheet (Machine Learning)

Model Evaluation is the process of measuring how well a Machine Learning model performs on unseen data.

Model Evaluation Workflow

```
Dataset
↓
Train-Test Split
↓
Train Model
↓
Predict
↓
Evaluate Performance
↓
Improve Model
```

1. Train-Test Split

Why?

- Training Data → Learn patterns
- Testing Data → Check performance

```
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(
    X,
    Y,
    test_size=0.2,
    random_state=42
)
```

Example:

- 80% Training
 - 20% Testing
-

2. Classification Evaluation Metrics

Used when output is categories:

- Yes/No
 - Spam/Not Spam
 - Disease/No Disease
-

Accuracy

Formula:

$$Accuracy = \frac{TP+TN}{TP+TN+FP+FN}$$

Python:

```
from sklearn.metrics import accuracy_score  
  
accuracy = accuracy_score(y_test, y_pred)  
  
print(accuracy)
```

Example:

100 predictions

90 correct

Accuracy = 90%

Precision

Measures correctness of positive predictions.

Formula:

$$Precision = \frac{TP}{TP+FP}$$

```
from sklearn.metrics import precision_score  
  
precision = precision_score(y_test, y_pred)
```

Example:

Predict 20 patients as sick

Actually sick = 15

Precision = 15/20 = 75%

Recall

Measures how many actual positives are found.

Formula:

$$Recall = \frac{TP}{TP+FN}$$

```
from sklearn.metrics import recall_score  
  
recall = recall_score(y_test, y_pred)
```

Example:

20 sick patients exist

Model found 18

Recall = 18/20 = 90%

F1 Score

Balance between Precision and Recall.

Formula:

$$F1 = 2 \times \frac{Precision \times Recall}{Precision + Recall}$$

```
from sklearn.metrics import f1_score  
  
f1 = f1_score(y_test, y_pred)
```

3. Confusion Matrix

Structure:

Actual/Predicted **Positive** **Negative**

Positive TP FN

Negative FP TN

Where:

- TP = True Positive

- TN = True Negative
 - FP = False Positive
 - FN = False Negative
-

Python Code

```
from sklearn.metrics import confusion_matrix

cm = confusion_matrix(y_test, y_pred)

print(cm)
```

Visualization

```
import seaborn as sns
import matplotlib.pyplot as plt

sns.heatmap(cm,
            annot=True,
            fmt='d')

plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.show()
```

4. Classification Report

Contains:

- Precision
- Recall
- F1 Score
- Support

```
from sklearn.metrics import classification_report

print(
    classification_report(
        y_test,
        y_pred
    )
)
```

5. Regression Evaluation Metrics

Used when output is numerical.

Examples:

- House Price
 - Salary Prediction
 - Temperature Prediction
-

Mean Absolute Error (MAE)

Formula:

$$MAE = \frac{1}{n} \sum |y - \hat{y}|$$

```
from sklearn.metrics import mean_absolute_error

mae = mean_absolute_error(
    y_test,
    y_pred
)
```

Meaning:

Average prediction error.

Mean Squared Error (MSE)

Formula:

$$MSE = \frac{1}{n} \sum (y - \hat{y})^2$$

```
from sklearn.metrics import mean_squared_error

mse = mean_squared_error(
    y_test,
    y_pred
)
```

Punishes large errors heavily.

Root Mean Squared Error (RMSE)

Formula:

$$RMSE = \sqrt{MSE}$$

```
import numpy as np

rmse = np.sqrt(mse)
```

R² Score

R² Score measures how well a regression model explains the variability of the target variable. It indicates the proportion of variance in the dependent variable that is predictable from the independent variables.

Formula:

$$R^2 = 1 - \frac{SS_{res}}{SS_{tot}}$$

```
from sklearn.metrics import r2_score

r2 = r2_score(
    y_test,
    y_pred
)
```

Interpretation:

R ²	Meaning
1	Perfect Model
0.9	Excellent
0.8	Good
0	Poor
Negative	Very Poor

6. Cross Validation

Cross Validation is a technique used to evaluate a machine learning model by dividing the dataset into multiple parts (folds) and training/testing the model multiple times. It helps determine how well the model will perform on unseen data.

Why Use Cross Validation?

- Reduces overfitting.
- Provides a more reliable performance estimate.
- Makes better use of limited data.

Check model stability.

```
from sklearn.model_selection import cross_val_score
from sklearn.tree import DecisionTreeClassifier

model = DecisionTreeClassifier()

scores = cross_val_score(
    model,
    X,
    y,
    cv=5
)

print(scores)
print(scores.mean())
```

7. Overfitting vs Underfitting

Overfitting

- Training Accuracy = 99%
- Testing Accuracy = 70%

Model memorized data.

Underfitting

- Training Accuracy = 50%
- Testing Accuracy = 45%

Model failed to learn.

Good Fit

- Training Accuracy = 90%
 - Testing Accuracy = 88%
-

Complete Classification Example

```
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier

from sklearn.metrics import (
    accuracy_score,
```

```

        precision_score,
        recall_score,
        f1_score,
        confusion_matrix,
        classification_report
    )

data = load_iris()

X = data.data
y = data.target

X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.2,
    random_state=42
)

model = RandomForestClassifier()

model.fit(X_train, y_train)

y_pred = model.predict(X_test)

print("Accuracy:",
      accuracy_score(y_test, y_pred))

print(classification_report(
    y_test,
    y_pred))

print(confusion_matrix(
    y_test,
    y_pred))

```

R² Score (Coefficient of Determination) – Definition

R² Score is a regression evaluation metric that measures how well the independent variables explain the variation in the dependent (target) variable.

It indicates the proportion of the variance in the target variable that is predictable from the input features.

Simple Definition

R² Score measures how well a regression model fits the data.

Interpretation

- **R² = 1** → Perfect prediction.
- **R² = 0** → Model explains none of the variability.
- **R² < 0** → Model performs worse than simply predicting the average.

Example

If a model has $R^2 = 0.85$, it means:

The model explains **85% of the variation** in the target variable, while the remaining 15% is unexplained.

Interview Questions

1. What is model evaluation?
2. Why do we split data into train and test sets?
3. What is accuracy?
4. What is precision?
5. What is recall?
6. What is F1 score?
7. What is a confusion matrix?
8. What is support in a classification report?
9. What is MAE?
10. What is MSE?
11. What is RMSE?
12. What is R^2 score?
13. What is cross-validation?
14. What is overfitting?
15. What is underfitting?
16. How do you detect overfitting?
17. Why is accuracy not always a good metric?
18. When should precision be prioritized?
19. When should recall be prioritized?
20. Why is F1 score useful?

Quick Revision

Classification

- Accuracy
- Precision
- Recall
- F1 Score
- Confusion Matrix
- Classification Report

Regression

- MAE
- MSE
- RMSE
- R^2 Score

Model Validation

- Train-Test Split
- Cross Validation
- Overfitting
- Underfitting